INSTITUTE OF ENGINEERING AND MANAGEMENT, SALTLAKE

School of

UNIVERSITY OF ENGINEERING AND MANAGEMENT, KOLKATA

ADVANCED PROGRAMMING (OOP) PROJECT

CODE – PCCCS495

PROJECT NAME: Expense Tracker App

NAME :	Sangram Keshari Patra
SECTION :	A
ROLL NUMBER :	33
ENROLLMENT NO. :	12024052017005
COURSE :	B. Tech
DEPARTMENT :	CSE (IOT CSBT)
SEMESTER :	4
YEAR :	2
SESSION :	SPRING 2026
INSTRUCTOR :	Prof. Swagatam Basu Prof. Soumadip Biswas Prof. Susovan Jana

Problem Statement

Managing personal finances is a common challenge faced by individuals across various walks of life, including students, freelancers, and professionals. Many people struggle to keep track of their daily expenses and incomes, leading to unintentional overspending and "budget leakage." Existing solutions often rely on cloud-based services, which may raise privacy concerns, require constant internet connectivity, or introduce unnecessary complexity for users who simply want a straightforward, secure, and offline tool.

There is a clear need for an intuitive, offline-first expense tracking application that empowers users to log transactions instantly, categorize their spending, and visualize their financial health without depending on external servers or risking data privacy. The solution should be easy to use, require minimal setup, and provide meaningful insights through dashboards and charts, helping users make informed financial decisions and develop better spending habits.

Target Users

- **Students:** Required to manage limited budgets, track daily expenses, and avoid overspending.
- **Freelancers:** Need a simple way to monitor irregular income streams and business expenses.
- **Professionals:** Want to organize personal and work-related finances efficiently and securely.

System Design

UML Class Diagram

Package Structure

```
com.expensetracker/
├── AppException.java
├── AppService.java
├── AppUtils.java
├── DatabaseManager.java
├── ExpenseTrackerApp.java
├── models/
│   ├── Category.java
│   ├── Transaction.java
│   ├── Income.java
│   ├── Expense.java
└── ui/
    ├── MainFrame.java
    ├── DashboardPanel.java
    ├── TransactionsPanel.java
    └── CategoriesPanel.java
```

Description of Java Classes

1. `AppException.java` - Handles all custom exceptions in the application. Used to provide meaningful error messages and manage error handling in a centralized way.
2. `AppService.java` - Acts as the service layer (ViewModel in MVVM). Contains business logic for managing transactions and categories, and mediates between the UI and the database.
3. `AppUtils.java` - Provides utility methods used throughout the application, such as date formatting, input validation, and other helper functions.
4. `DatabaseManager.java` - Responsible for all database operations. Manages the SQLite connection, executes CRUD operations for transactions and categories, and ensures data persistence.
5. `ExpenseTrackerApp.java` - The main entry point of the application. Initializes the main window (MainFrame) and starts the application.
6. `models/Category.java` - Represents a financial category (e.g., Food, Travel). Contains fields for category name and any relevant metadata.
7. `models/Transaction.java` - Represents a financial transaction. Contains fields for amount, date, category, description, and type (income/expense).
8. `models/Income.java` - Represents an income transaction. May extend Transaction or encapsulate additional fields specific to income.
9. `models/Expense.java` - Represents an expense transaction. May extend Transaction or encapsulate additional fields specific to expenses.
10. `ui/MainFrame.java` - The main application window. Contains a tabbed interface to switch between Dashboard, Transactions, and Categories panels.
11. `ui/DashboardPanel.java` - Displays charts and statistics summarizing the user's financial data, such as total balance and spending by category.
12. `ui/TransactionsPanel.java` - Allows users to add, edit, delete, and view transactions. Provides filtering and search functionality.
13. `ui/CategoriesPanel.java` - Allows users to manage categories, including adding new categories and editing or deleting existing ones.

Interactive Flow

1. **Application Launch**
   - The user runs the application via ExpenseTrackerApp.java.
   - MainFrame is initialized and displayed.
2. **User Interface**
   - The main window (MainFrame) presents three tabs: Dashboard, Transactions, and Categories.
   - Each tab is managed by its respective panel class (DashboardPanel, TransactionsPanel, CategoriesPanel).
3. **User Actions**
   - Dashboard Tab: User views charts and statistics. Data is fetched from AppService, which retrieves it from DatabaseManager.
   - Transactions Tab: User adds, edits, or deletes transactions. Actions are sent to AppService, which updates the database via DatabaseManager and refreshes the UI.
   - Categories Tab: User manages categories. Changes are processed by AppService and persisted by DatabaseManager.
4. **Data Flow**
   - UI panels interact with AppService for all business logic.
   - AppService calls DatabaseManager for data operations.
   - DatabaseManager performs CRUD operations on the SQLite database.
   - Results are returned up the chain to update the UI.
5. **Error Handling**
   - Any errors encountered in the service or database layers are thrown as AppException.
   - UI panels catch these exceptions and display user-friendly error messages.

OOP Concepts Demonstrated

Abstraction

DatabaseManager as DAO

The DatabaseManager class provides high-level methods for database operations, hiding the underlying SQL queries from the rest of the application.

```
// DatabaseManager.java
public class DatabaseManager {
    public List<Transaction> getAllTransactions() throws AppException {
        // ... SQL logic hidden from caller ...
    }
    public void insertTransaction(Transaction t) throws AppException {
        // ... SQL logic hidden from caller ...
    }
}
```

AppService as Service Layer

AppService abstracts business logic, so UI classes do not need to know how data is managed or stored.

```
// AppService.java
public class AppService {
    private DatabaseManager dbManager;
    public void addTransaction(Transaction t) throws AppException {
        dbManager.insertTransaction(t);
    }
    public List<Transaction> getTransactions() throws AppException {
        return dbManager.getAllTransactions();
    }
}
```

Inheritance

UI Panels Extending JPanel

All UI panels inherit from `JPanel`, gaining all its methods and properties, and can be used as Swing components.

```
// DashboardPanel.java
public class DashboardPanel extends JPanel {
    // Custom dashboard UI
}
// TransactionsPanel.java
public class TransactionsPanel extends JPanel {
    // Custom transactions UI
}
```

Expense and Income Extending Transaction

Expense and Income inherit from Transaction, reusing common fields and methods.

```
// Transaction.java
public class Transaction {
    protected double amount;
    protected String category;
    // ...
}
// Expense.java
public class Expense extends Transaction {
    private String expenseType;
    // ...
}
// Income.java
public class Income extends Transaction {
    private String source;
    // ...
}
```

Polymorphism

Overriding paintComponent

Each panel can override paintComponent to render its own content, but can be referenced as a JPanel.

```
// DashboardPanel.java
@Override
public void paintComponent(Graphics g) {
    super.paintComponent(g);
    // Custom drawing code for dashboard
}
// TransactionsPanel.java
@Override
public void paintComponent(Graphics g) {
    super.paintComponent(g);
    // Custom drawing code for transactions
}
```

Handling Transactions Polymorphically

Both `Expense` and `Income` can be treated as `Transaction` objects, allowing uniform processing.

```
List<Transaction> transactions = new ArrayList<>();
transactions.add(new Expense(...));
transactions.add(new Income(...));
for (Transaction t : transactions) {
    System.out.println(t.getAmount());
}
```

Exception Handling

Custom Exception

```
// AppException.java
public class AppException extends Exception {
    public AppException(String message) {
        super(message);
    }
}
```

Handling Database Errors

Custom exceptions are thrown and caught to provide user-friendly error messages.

```
try {
    dbManager.insertTransaction(t);
} catch (AppException e) {
    JOptionPane.showMessageDialog(this, "Error: " + e.getMessage());
}
```

Handling NullPointerException in UI

UI code handles potential null values to prevent application crashes.

```
try {
    String value = someTextField.getText();
    // ... use value ...
} catch (NullPointerException e) {
    JOptionPane.showMessageDialog(this, "Unexpected error occurred.");
}
```

Collections

Using List for Transactions

A `List` is used to store and iterate over transactions.

```
List<Transaction> transactions = dbManager.getAllTransactions();
for (Transaction t : transactions) {
    System.out.println(t.getAmount());
}
```

Using Map for Category Totals

A `Map` is used to aggregate totals by category for chart visualization.

```
Map<String, Double> categoryTotals = new HashMap<>();
for (Transaction t : transactions) {
    categoryTotals.put(
        t.getCategory(),
        categoryTotals.getOrDefault(t.getCategory(), 0.0) + t.getAmount()
    );
}
```

Threading

Running Database Operations Off the UI Thread

Database operations are performed in a separate thread to avoid freezing the UI, and UI updates are posted back to the Event Dispatch Thread.

```
// In TransactionsPanel.java
new Thread(() -> {
    try {
        dbManager.insertTransaction(t);
        SwingUtilities.invokeLater(() -> {
            // Update UI after DB operation
        });
    } catch (AppException e) {
        SwingUtilities.invokeLater(() -> {
            JOptionPane.showMessageDialog(this, "Error: " + e.getMessage());
        });
    }
}).start();
```

Implementation Highlights

This project demonstrates clean class structures in intelij format, robust error handling, and efficient data management. Some code snippets illustrating important implementation aspects are:

Adding a Transaction via Service Layer

This method encapsulates the logic for adding a transaction, ensuring the UI does not interact directly with the database. It promotes maintainability and abstraction.

```
// AppService.java
public void addTransaction(Transaction t) throws AppException {
    dbManager.insertTransaction(t);
}
```

Fetching and Displaying Transactions

Transactions are fetched from the service layer and displayed in a table. This demonstrates the use of collections and separation between data retrieval and UI rendering.

```
// TransactionsPanel.java
List<Transaction> transactions = appService.getTransactions();
for (Transaction t : transactions) {
    tableModel.addRow(new Object[]{t.getDate(), t.getCategory(),
t.getAmount()});
}
```

Exception Handling in the UI

All errors are caught and shown to the user in a dialog, ensuring the application remains user-friendly and robust against failures.

```
try {
    appService.addTransaction(newTransaction);
} catch (AppException e) {
    JOptionPane.showMessageDialog(this, "Error: " + e.getMessage());
}
```

Calculating Category Totals for Visualization

This snippet aggregates transaction amounts by category, enabling the generation of pie or bar charts for the dashboard.

```
Map<String, Double> categoryTotals = new HashMap<>();
for (Transaction t : transactions) {
    categoryTotals.put(
        t.getCategory(),
        categoryTotals.getOrDefault(t.getCategory(), 0.0) + t.getAmount()
    );
}
```

Testing & Error Handling

Testing

- The application was tested through a combination of manual and functional tests to ensure reliability and correctness. Key test cases included:
- Adding, editing, and deleting transactions and categories.
- Verifying correct calculation of balances and category totals.
- Handling empty input fields and invalid data (e.g., negative amounts).
- Ensuring the application creates the SQLite database file on first run.
- Checking UI responsiveness during database operations.

Edge cases, such as duplicate category names, empty transaction lists, and invalid date formats, were also considered to ensure robust behavior.

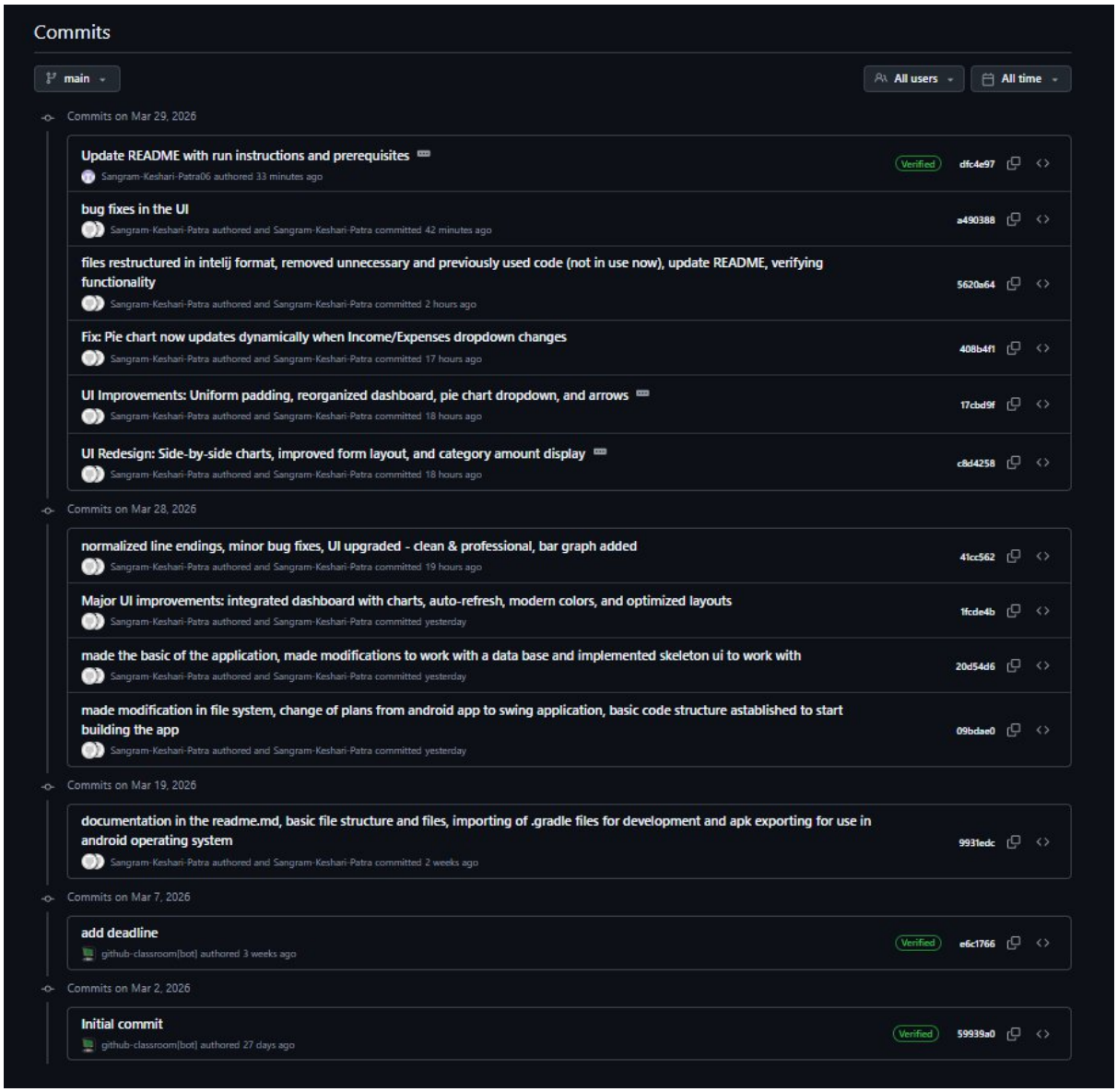
Error Handling:

- Robust error handling is implemented throughout the application:
- All database and business logic errors are wrapped in custom AppException objects.
- The UI catches exceptions and displays user-friendly error messages using dialog boxes, preventing application crashes.
- Try-catch blocks are used around critical operations, such as database access and user input processing.
- The application exits gracefully on unrecoverable errors, ensuring data integrity and a good user experience.

This approach ensures that users are informed of issues in a clear manner and that the application remains stable even in unexpected scenarios.

Git Workflow

Commit History:



Development Progression:

1. **Initial project setup**
   Set up the basic Maven project structure and initialized the repository.
2. **Add model classes**
   Implemented core data models for transactions, categories, income, and expenses.
3. **Implement DatabaseManager**
   Added database access layer for CRUD operations using SQLite.
4. **Create AppService layer**
   Developed service layer to handle business logic and mediate between UI and database.
5. **Build main UI panels**
   Designed DashboardPanel, TransactionsPanel, and CategoriesPanel for the main interface.

6. **Integrate charts in dashboard**
   Added pie and bar chart visualizations to the dashboard for spending analysis.
7. **Implement error handling**
   Introduced custom exceptions and user-friendly error dialogs.
8. **Add transaction filtering**
   Enabled filtering of transactions by date and amount in the UI.
9. **Refactor and optimize code**
   Cleaned up redundant code, improved method names, and enhanced performance.
10. **Update README and documentation**
    Provided detailed instructions for setup, usage, and project structure.